

Custom Partition Table

I. Experiment Objective

Learn the basic concepts and configuration methods of the ESP32 custom partition table. Through this experiment, you will understand the structure and function of the partition table and master how to customize the Flash partition layout for the MicroROS-V2 control board.

II. Experiment Principle

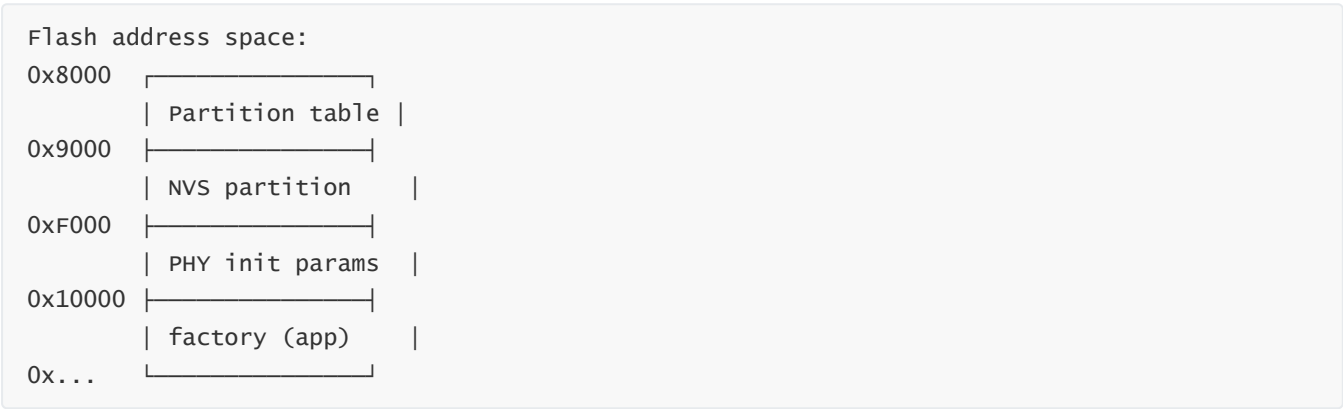
1. Keywords

Keyword	Description
Partition Table	The layout map of ESP32 Flash storage, defining the name, type, offset address, and size of each partition. It is stored at offset 0x8000 in Flash.
NVS partition	Non-Volatile Storage partition, used to save parameters that must not be lost after power-off, such as Wi-Fi configuration and calibration data.
Application partition (app)	The partition that stores the firmware code. ESP-IDF uses the factory partition as the application partition by default.
Offset address	The start address of a partition in Flash. Each partition address must be aligned to 0x1000 (4KB).
OTA upgrade	A mechanism for wireless firmware upgrade by adding two application partitions, ota_0 and ota_1.

2. Technical Architecture

The Flash space of the ESP32 is managed uniformly through the partition table. The partition table file (partitions.csv) is located in the project root directory and is converted to binary format during compilation and flashed to offset 0x8000 in Flash. At system startup, the Bootloader loads the corresponding application partition according to the partition table.

Partition table layout:



III. Experiment Steps

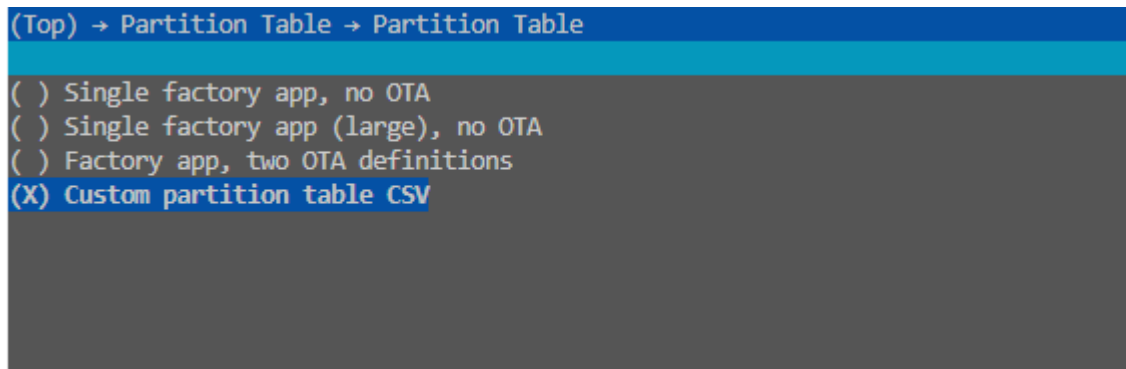
1. Hardware Preparation

Hardware	Requirement
MicroROS-V2 control board	Required
USB data cable	Required

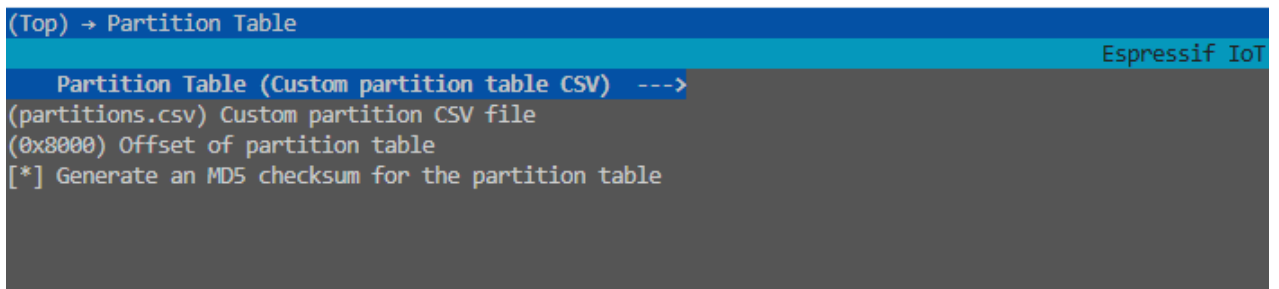
2. Prerequisites

1. Set up the ESP32-IDF development environment
2. Connect the MicroROS-V2 control board to the computer via USB
3. Open the IDF configuration tool and specify partitions.csv as the partition table.

```
cd ~/esp/base_samples/partition_table
idf.py menuconfig
```



```
(Top) → Partition Table → Partition Table
( ) Single factory app, no OTA
( ) Single factory app (large), no OTA
( ) Factory app, two OTA definitions
(X) Custom partition table CSV
```



```
(Top) → Partition Table
Partition Table (Custom partition table CSV) --->
(partitions.csv) Custom partition CSV file
(0x8000) Offset of partition table
[*] Generate an MD5 checksum for the partition table
```

3. Compile and Flash

```
cd ~/esp/base_samples/partition_table
source ~/esp/esp-idf/export.sh
idf.py build flash monitor
```

4. Stopping Method

In the terminal, press `Ctrl+]` to exit the serial monitor.

IV. Experiment Result

After successful compilation and flashing, the Monitor window will output the following:

```
hello yahboom
I (xxx) MAIN: Nice to meet you!
```

The normal startup of the control board indicates that the firmware has been correctly flashed and successfully booted through the custom partition table.

V. Program Analysis

Source code paths:

- Firmware source: `~/esp/base_samples/partition_table/main/main.c`
- Partition table: `~/esp/base_samples/partition_table/partitions.csv`

1. Partition Table Configuration File

Parameter definition file: `~/esp/base_samples/partition_table/partitions.csv`

The partition table is defined in the `partitions.csv` file in the project root directory, with the following content:

```
# Name,    Type, SubType, Offset,  Size,   Flags
nvs,      data, nvs,     0x9000, 0x6000,
phy_init, data, phy,     0xf000, 0x1000,
factory,  app,  factory, 0x10000, 3M,
```

This partition table defines three partitions:

Partition Name	Type	SubType	Offset	Size	Description
nvs	data	nvs	0x9000	0x6000 (24KB)	Non-volatile storage, used to save Wi-Fi configuration, calibration data, etc.
phy_init	data	phy	0xf000	0x1000 (4KB)	PHY initialization parameters
factory	app	factory	0x10000	3MB	Application partition, stores the firmware code

The first two records of the partition table (nvs and phy_init) are system-required partitions. The `factory` partition after the third record is the application partition.

2. Main Program Logic

`main/main.c` is the sample main program. It only prints a welcome message to verify that the firmware has been correctly flashed to the custom partition:

```
#include <stdio.h>
#include "esp_log.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

static const char *TAG = "MAIN";

void app_main(void)
{
    printf("hello yahboom\n");
    ESP_LOGI(TAG, "Nice to meet you!");
}
```

3. The Role of the Partition Table

The partition table is the layout map of ESP32 Flash storage. It defines the name, type, offset address, and size of each partition. By default, ESP-IDF uses the factory default partition table. When the application needs a larger firmware space, OTA upgrade functionality, or custom data storage, the partition table must be modified.

Through a custom partition table, you can:

- **Adjust the firmware size:** enlarge the app partition to accommodate a larger firmware
- **OTA upgrade:** add two app partitions, `ota_0` and `ota_1`, to implement wireless upgrades
- **Custom storage partitions:** add file system partitions such as `fat` and `spiffs` to store user data
- **Optimize the layout:** adjust the size allocation of each partition according to actual needs

4. SDK Configuration File Description

The sample directory also contains the `sdkconfig.baiduyun.uploading.cfg` file, which is a preconfigured SDK configuration file used for the Baidu Netdisk firmware flashing scenario. When you build with `idf.py build`, your own `sdkconfig` file is generated.

VI. Common Problems

Problem	Cause	Solution
Cannot start after flashing	The partition table does not match the firmware size	Check whether the app partition size is large enough to hold the compiled firmware
NVS initialization fails	The NVS partition is corrupted or the version does not match	Run <code>idf.py erase_flash</code> to erase the entire Flash

Problem	Cause	Solution
Partition table format error	The CSV format is incorrect	Ensure the CSV file uses commas as separators and that the partition name, type, and subtype are correct